

Spring Security + JWT 기술을 이용한 BackEnd 실습

: BackEnd 완성 후 향후 React기술을 이용하여 FrontEnd 와 연결 할 예정이다.

☞주요기능

-회원관리

- 1) 회원가입
- 2) 로그인- Spring Security를 이용한 인증(JWT방식)
- 3) 로그아웃 - 세션 invalidate 진행 X, 로컬스토리지 에서 토큰을 삭제O
(실제 서버상 코드가 필요없다.)

인증된 사용자 | 인증되지 않은 사용자가 서비스 요청시 권한을 제한하는 코드 필요

-게시판 관리

- 1) 전체검색 -누구나 사용
- 2) 글 등록 - 인증된 사용자만
- 3) 상세보기 - 인증된 사용자만
- 4) 수정 - 인증된 사용자만
- 5) 삭제 - 인증된 사용자만

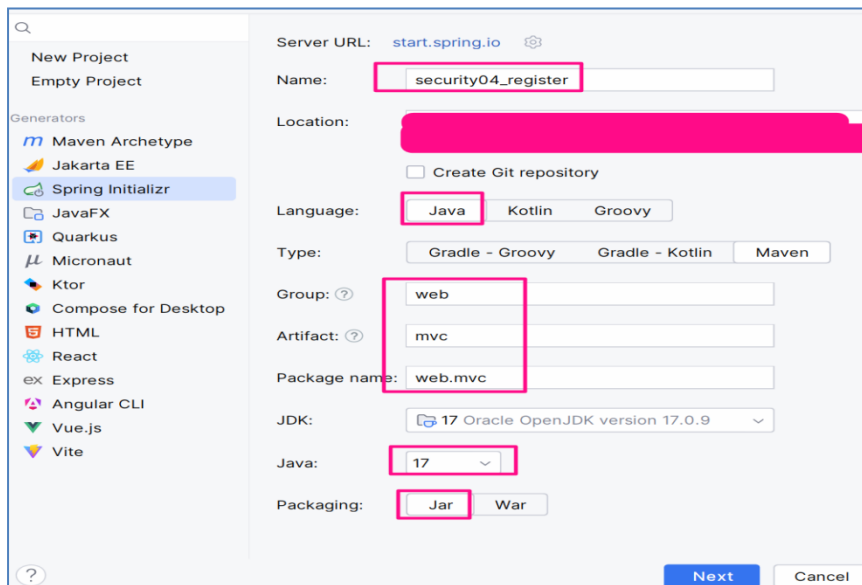
-관리자 - ROLE_ADMIN 인경우만

Project 4- security04_Register

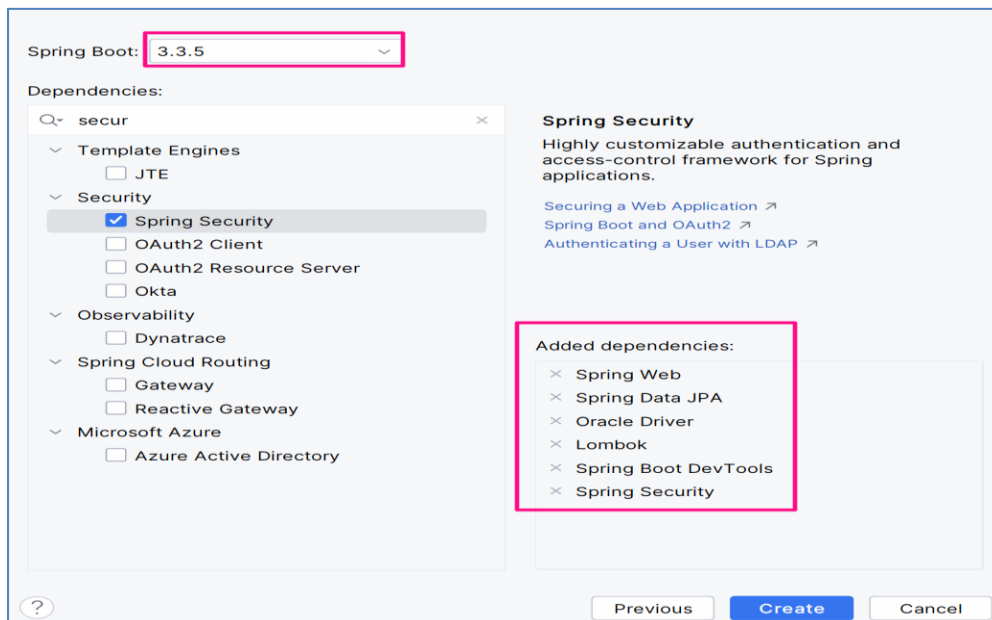
Group ID : web

Artifact ID :mvc

Dependency : Spring Security, Spring Web, Lombok, mysql , JPA , Dev Tool,

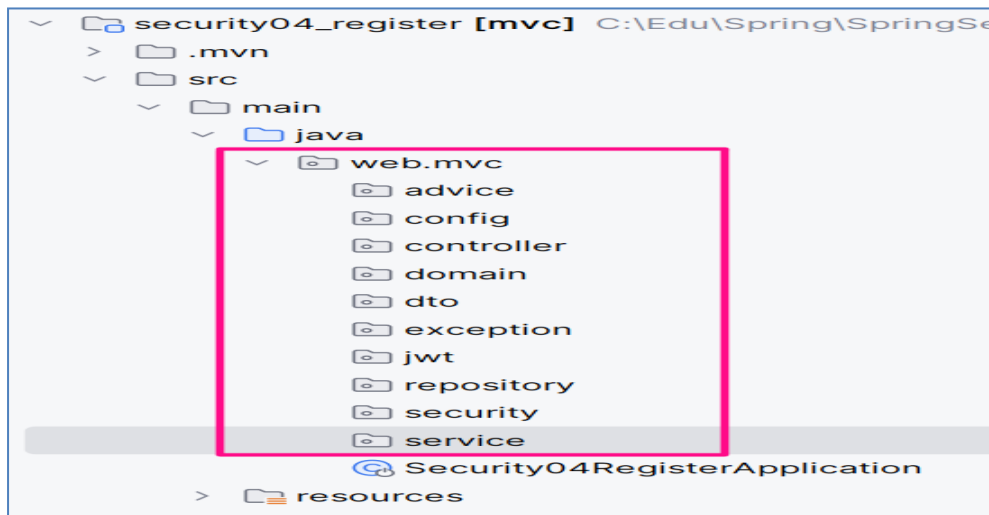


Spring Initializr project configuration window. The 'Name' field is set to 'security04_register'. The 'Language' is set to 'Java'. The 'Type' is set to 'Gradle - Groovy'. The 'Group' is set to 'web', the 'Artifact' is set to 'mvc', and the 'Package name' is set to 'web.mvc'. The 'JDK' is set to '17 Oracle OpenJDK version 17.0.9'. The 'Packaging' is set to 'Jar'. The 'Next' button is highlighted.



Spring Boot dependencies selection window. The 'Spring Boot' version is set to '3.3.5'. The 'Dependencies' section shows 'Spring Security' selected. The 'Added dependencies' list includes: Spring Web, Spring Data JPA, Oracle Driver, Lombok, Spring Boot DevTools, and Spring Security. The 'Create' button is highlighted.

📁 프로젝트 directory 구조



☞ 기본 세팅 상태에서 프로젝트 구동해보자.

프로젝트를 생성한 후

1) Boot version 3.5.0

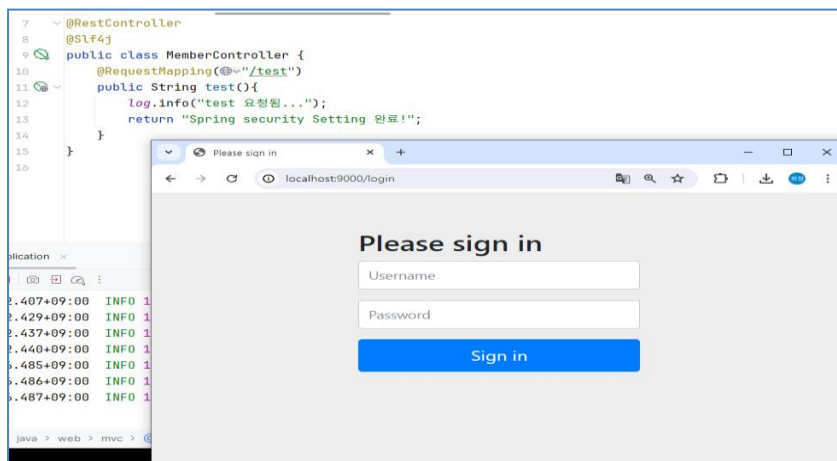
2) **main**클래스에 설정

@SpringBootApplication(exclude = DataSourceAutoConfiguration.class)

3) application.properties파일에 port번호 수정한다.

4) RestController를 요청 할 컨트롤러를 하나 만든다.

5) 브라우저에서 요청해본다. (ex) `http://localhost:9000/test`



세팅을 완료 후에 브라우저에서 `http://localhost:9000/test`

요청하면 아래와 같은 **spring security** 내에서 제공하는 인증 폼이 나온다.

☞ **회원관리 - 회원가입 및 아이디중복체크 구현**

1) DB 준비

@SpringBootApplication(exclude = DataSourceAutoConfiguration.class) 주석풀기

properties 파일에 설정 추가

```
spring.datasource.driver-class-name=com.mysql.cj.jdbc.Driver
spring.datasource.url=jdbc:mysql://localhost:3307/jpa?serverTimezone=Asia/Seoul
spring.datasource.username=jang
spring.datasource.password=jang1234

spring.jpa.database-platform=org.hibernate.dialect.MySQL8Dialect
spring.jpa.hibernate.ddl-auto=none
spring.jpa.generate-ddl=false
spring.jpa.show-sql=true
spring.jpa.database=mysql
```

Entity만들기 - Member.java

```
9      @Entity
10      @Getter
11      @Setter
12      @Builder
13      @NoArgsConstructor
14      @AllArgsConstructor
15      public class Member {
16          @Id
17          @GeneratedValue(strategy = GenerationType.IDENTITY)
18          private Long memberNo ;
19
20          @Column(unique = true)
21          private String id;
22          private String pwd;
23
24          @Column(length = 20)
25          private String name;
26          private String address;
27
28          private String role;
29
30          @CreationTimestamp
31          private LocalDateTime regDate;
32
33      }
```

Entity만들기 - Board.java

```

10  @AllArgsConstructor
11  @NoArgsConstructor
12  @Setter
13  @Getter
14  @ToString
15  @Entity // 서버 실행시에 해당 객체로 테이블 매핑생성
16  @Builder
17  public class Board {
18      @Id//pk를 해당 필드로 한다
19      @GeneratedValue(strategy = GenerationType.IDENTITY)
20      private Long id;//글번호
21
22      private String title;//제목
23
24      @Column(length =100)
25      private String content;//내용
26
27
28      @ManyToOne(fetch = FetchType.LAZY)
29      @JoinColumn(name ="member_no")
30      private Member member;//작성자
31
32      @CreationTimestamp
33      private LocalDateTime regDate;//등록일
34
35      @UpdateTimestamp
36      private LocalDateTime updateDate;//수정일
37  }
38

```

→서버 restart 한 후 테이블이 생성된 것을 확인하고 설정의 DDL 옵션을 none변경 한다.

```

Hibernate: drop table if exists board
Hibernate: drop table if exists member
Hibernate: create table board (id bigint not null auto_increment, member_no bigint, reg_date datetime(6), update_date datetime(6), content varchar(100), title varchar(255), primary key (id)) engine=InnoDB
Hibernate: create table member (member_no bigint not null auto_increment, reg_date datetime(6), name varchar(20), address varchar(255), id varchar(255), pwd varchar(255), role varchar(255), primary key (member_no)) engine=InnoDB
Hibernate: alter table member add constraint UKjp8ds32yg1s0sxw2rkiagn768 unique (id)
Hibernate: alter table board add constraint FKbuilrr94cdul9mly722loe7jp foreign key (member_no) references member (member_no)
2025-06-11T20:21:17.026+09:00 INFO 10032 --- [security04_register] [ restartedMain ] j.LocalContainerEntityManagerFactoryBean : Initialized JPA EntityManagerFactory for persistence unit 'default'

```

스프링 시큐리티 인가 및 설정을 담당하는 SecurityConfig.java 만들기

: 스프링 버전이 업데이트 되면서 설정하는 방법이나 사용 할 수 있는 메소드들이 다르다

1) 스프링 부트 2.6 : 스프링 5.x

- WebSecurityConfigurerAdapter 상속 받고 메소드 재정의

```

@Configuration
@EnableWebSecurity
public class SecurityConfig extends WebSecurityConfigurerAdapter{

    @Override
    protected void configure(HttpSecurity http) throws Exception {

        http
            .authorizeRequests() // security-context <security:intercept-url
            .antMatchers("/user/login") // pattern="/member/main"
            .hasRole("USER") // access="isAuthenticated()"
            .antMatchers("/member/**")
            .authenticated()
            .antMatchers("/admin/**")
            .hasRole("ADMIN")
            .and()
            //.csrf().disable() // <security:csrf disabled="true"/>
            .formLogin()
            .loginPage("/user/loginForm")
            .loginProcessingUrl("/loginCheck")
            .usernameParameter("id")
            .passwordParameter("pwd")
            .defaultSuccessUrl("/")
            .failureForwardUrl("/user/loginForm?err")
            .and()
            .logout()
            .logoutUrl("/logout")
            .logoutSuccessUrl("/")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
            .and();
    }
}

```

2) 스프링 부트 2.7 ~ 3.0 : 스프링 5.7 ~ 6.x)

- 상속 받지 않고 SecurityFilterChain를 빈으로 등록한다.

```

@Configuration
@EnableWebSecurity
public class SecurityConfig2 {
    @Bean
    protected SecurityFilterChain configure(HttpSecurity http) throws Exception {
        http
            .authorizeRequests() // security-context <security:intercept-url
            .antMatchers("/user/login") // pattern="/member/main"
            .hasRole("USER") // access="isAuthenticated()"
            .antMatchers("/member/**")
            .authenticated()
            .antMatchers("/admin/**")
            .hasRole("ADMIN")
            .and()
            //.csrf().disable() // <security:csrf disabled="true"/>
            .formLogin()
            .loginPage("/user/loginForm")
            .loginProcessingUrl("/loginCheck")
            .usernameParameter("id")
            .passwordParameter("pwd")
            .defaultSuccessUrl("/")
            .failureForwardUrl("/user/loginForm?err")
            .and()
            .logout()
            .logoutUrl("/logout")
            .logoutSuccessUrl("/")
            .invalidateHttpSession(true)
            .deleteCookies("JSESSIONID")
            .and();

        return http.build();
    }
}

```

3) 스프링 부트 3.1 : 스프링 6.1

: 람다형식으로 변경됨.

web/mvc/config/SecurityConfig.java

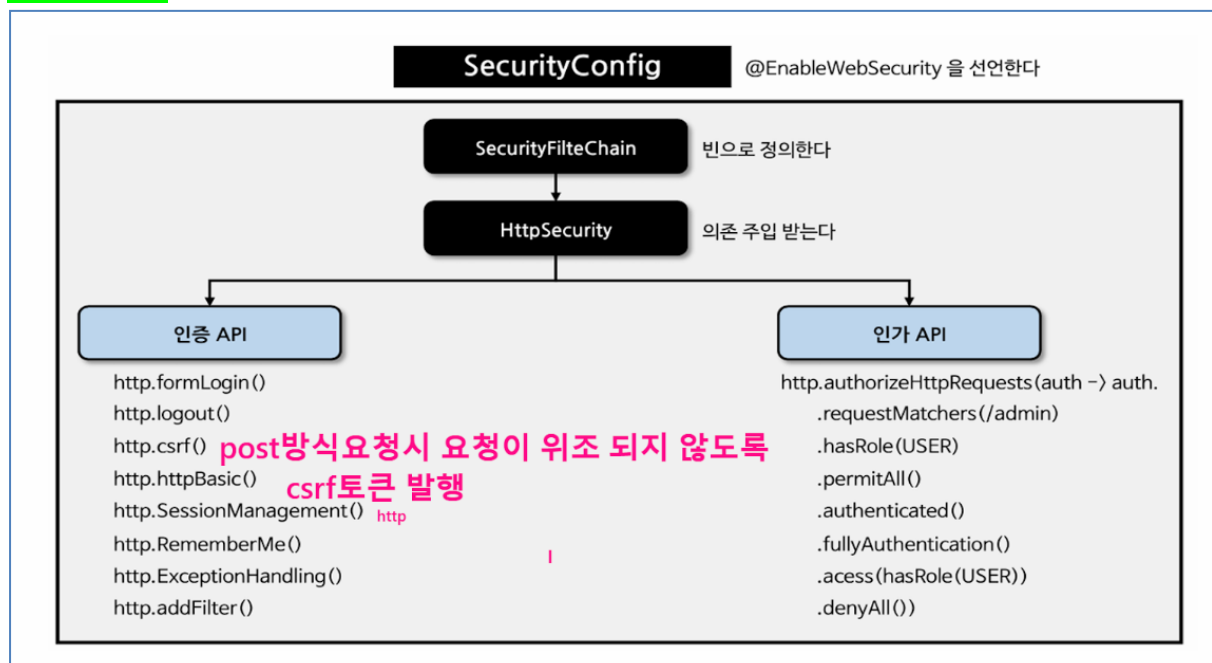
```

16 @Configuration
17 @EnableWebSecurity
18 @Slf4j
19 public class SecurityConfig {
20
21     @Bean
22     public BCryptPasswordEncoder bCryptPasswordEncoder() {
23         log.info("BCryptPasswordEncoder call.....");
24         return new BCryptPasswordEncoder();
25     }
26
27
28     @Bean
29     public SecurityFilterChain filterChain(HttpSecurity http) throws Exception {
30         log.info("SecurityFilterChain filterChain(HttpSecurity http) call.....");
31         ///////////////////////////////////////////////////
32         //csrf disable
33         http.csrf((auth) -> auth.disable()); //csrf공격을 방어하기 위한 토큰 주고 받는 부분을 비활성화!
34
35         //Form 로그인 방식 disable -> React, JWT 인증 방식으로 변경예정
36         //disable 를 설정하면 시큐리티의 UsernamePasswordAuthenticationFilter비활성됨.
37         http.formLogin((auth) -> auth.disable());
38
39         //http basic 인증 방식 disable
40         http.httpBasic((auth) -> auth.disable());
41
42         //경로별 인가 작업
43         http.authorizeHttpRequests((auth) ->
44             auth
45                 .requestMatchers("/index", "/members", "/members/**", "/boards").permitAll()
46                 .requestMatchers("/admin").hasRole("ADMIN")
47                 .anyRequest().authenticated());
48
49         return http.build();
50     }
51 }

```

SecurityFilterChain - 우리가 원하는 시큐리티 정책 선언

HttpSecurity - 어떤 요청은 인증작업을 실행한다/ 안한다 하는 등의 정보를 세팅 하는 것



chain.getFilters().forEach(System.out::println); //내장하고 있는
기본 정책의 필터객체들을 확인해보자.



```
org.springframework.security.web.session.DisableEncodeUrlFilter@6fff46bf
org.springframework.security.web.context.request.async.WebAsyncManagerIntegrationFilter@1835dc92
org.springframework.security.web.context.SecurityContextHolderFilter@45c9b3
org.springframework.security.web.header.HeaderWriterFilter@3a6045c6
org.springframework.security.web.csrf.CsrfFilter@3234474
org.springframework.security.web.authentication.logout.LogoutFilter@25f15f50
org.springframework.security.web.authentication.UsernamePasswordAuthenticationFilter@1d8b0500
DefaultResourcesFilter [matcher=Ant [pattern='/default-ui.css', GET], resource=org/springframework/security/default-ui.css]
org.springframework.security.web.authentication.ui.DefaultLoginPageGeneratingFilter@d3f4505
org.springframework.security.web.authentication.ui.DefaultLogoutPageGeneratingFilter@3aaa3c39
org.springframework.security.web.savedrequest.RequestCacheAwareFilter@6680f714
org.springframework.security.web.servletapi.SecurityContextHolderAwareRequestFilter@5a936e64
org.springframework.security.web.authentication AnonymousAuthenticationFilter@65a9ea3c
org.springframework.security.web.access.ExceptionTranslationFilter@63300c4b
org.springframework.security.web.access.intercept.AuthorizationFilter@748a654a
```

BCryptPasswordEncoder는 **Spring Security**에서 비밀번호를 안전하게 암호화하기 위한 클래스로, 주로 사용자의 비밀번호를 데이터베이스에 안전하게 저장하고 비교할 때 사용된다.

BCryptPasswordEncoder는 **BCrypt**해싱 알고리즘을 기반으로 작동하며, 이 알고리즘은 암호화된 비밀번호를 해독하기 어렵도록 설계되어 있다.

예시) 단위 test 해보기


```

20
21 @Autowired
22 private PasswordEncoder passwordEncoder;
23 /**
24  * 패스워드 암호화 테스트
25  * */
26 @Test
27 @DisplayName("암호화 test")
28 void passwordTest(){
29     String rawPassword="8253jang";
30
31     // 비밀번호 인코딩
32     String encodedPassword = passwordEncoder.encode(rawPassword); // 평문 -> 암호화
33     log.info("encodedPassword = {}", encodedPassword);
34
35     // 비밀번호 매칭 확인
36     boolean isPasswordMatch = passwordEncoder.matches(rawPassword, encodedPassword);
37     log.info("Password match : {}", isPasswordMatch);
38 }

```

결과)

```

Security04Reg 827 ms Tests passed: 1 of 1 test - 827 ms
암호화 test 827 ms
s because bootstrap classpath has been appended
curity04RegisterApplicationTests : encodedPassword = $2a$10$e8r314o7est1prpMYc00L.zZbIE/aQLC12oP3Ci78H.k.fyQV.xLm
curity04RegisterApplicationTests : Password match : true

```

Repository 만들기 - MemberRepository.java

```

4 public interface MemberRepository extends JpaRepository<Member, Long>{
5
6     @Query("select m from Member m where m.id=?1")
7     Member duplicateCheck(String id);
8
9     Boolean existsById(String id);
10
11 | Member findById(String id);
12
13
14 }

```

Service 만들기 - MeberService.java

```

6 public interface MemberService {
7
8     String duplicateCheck(String id);
9
10    /**
11     * 가입
12     * */
13    void signUp(Member member);
14
15 }

```

Service 만들기 - MemberServiceImpl.java

```

14 @Service
15 @RequiredArgsConstructor
16 @Slf4j
17 public class MemberServiceImpl implements MemberService {
18     private final MemberRepository memberRepository;
19     private final PasswordEncoder passwordEncoder; → 비밀번호 암호화 하기 위한 주입
20
21     @Transactional(readOnly = true)
22     @Override
23     public String duplicateCheck(String id) {
24         Member member = memberRepository.duplicateCheck(id);
25         System.out.println("member = " + member);
26         if(member==null) return "사용가능합니다.";
27         else return "중복입니다.";
28     }
29
30     @Transactional
31     @Override
32     public void signUp(Member member) {
33         if(memberRepository.existsById(member.getId())){
34             throw new MemberAuthenticationException(ErrorCode.DUPLICATED);
35         }
36         member.setPwd(passwordEncoder.encode(member.getPwd()));
37         member.setRole("ROLE_USER");
38
39         Member m = memberRepository.save(member); //동일한 id가 들어오면 수정됨
40         log.info("m = {}", m);
41     }
42 }

```

Exception 만들기

-ErrorCode.java

```

3 public enum ErrorCode { //enum은 'Enumeration' 의 약자로 열거, 목록 이라는 뜻
4
5     DUPLICATED(HttpStatus.BAD_REQUEST, "Duplicate Id", "아이디가 중복입니다."),
6     WRONG_PASS(HttpStatus.BAD_REQUEST, "password wrong", "비밀번호 오류입니다."),
7
8     NOTFOUND_NO(HttpStatus.NOT_FOUND, "Not Found Board SearchById", "글번호를 확인하세요."),
9     NOTFOUND_BOARD(HttpStatus.BAD_REQUEST, "Not Found Board All", "전체 게시물을 조회 할수 없습니다."),
10
11     UPDATE_FAILED(HttpStatus.BAD_REQUEST, "Update fail", "수정할수 없습니다."),
12     DELETE_FAILED(HttpStatus.BAD_REQUEST, "Delete fail", "삭제할 수 없습니다.");
13
14     private final HttpStatus httpStatus;
15     private final String title;
16     private final String message;
17 }

```

-MemberAuthenticationException.java

```

7      @Getter 5 usages
8      @RequiredArgsConstructor
9      public class MemberAuthenticationException extends RuntimeException{
10         private final ErrorCode errorCode;
11     }
12

```

DefaultExceptionAdvice.java : @RestControllerAdvice

```

3  @RestControllerAdvice
4  public class DefaultExceptionAdvice {
5      @ExceptionHandler({MemberAuthenticationException.class})
6      public ProblemDetail signInExceptionHandler(MemberAuthenticationException e){
7          ProblemDetail problemDetail = ProblemDetail.forStatus(e.getErrorCode().getHttpStatus());
8
9          problemDetail.setTitle(e.getErrorCode().getTitle());
10         problemDetail.setDetail(e.getErrorCode().getMessage());
11         problemDetail.setProperty("timestamp", LocalDateTime.now());
12
13     return problemDetail;
14     }
15
16 }

```

 **ProblemDetails에 대하여 첨부된 교안자료 참고(05_Springboot3_ProblemDetail정리.pdf)**

Controller 만들기 – MeberController.java

```

8
9  @RequiredArgsConstructor
10  @RestController
11  @Slf4j
12  public class MemberController {
13      private final MemberService memberService;
14
15      @GetMapping("/index")
16      public String index(){
17          log.info("index 요청됨...");
18          return "Spring security Setting 완료!";
19      }
20
21      /**
22       * 아이디 중복체크
23       * */
24      @GetMapping("/members/{id}")
25      public String duplicateIdCheck(@PathVariable String id){
26          log.info("id : {}", id);
27          return memberService.duplicateCheck(id);
28      }
29
30      /**
31       * 회원가입
32       * */
33      @PostMapping("/members")
34      public String signUp(@RequestBody Member member){
35          memberService.signUp(member);
36          return "ok";
37      }
38  }
39

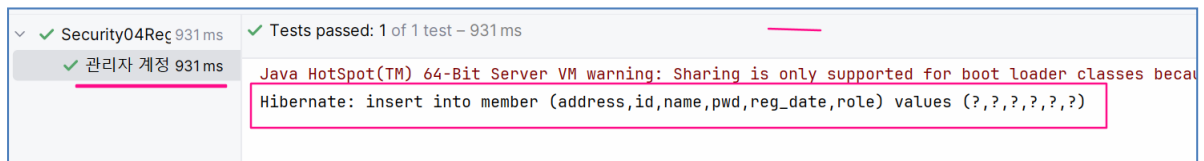
```

Test 만들기 – test > java > web > mvc > 관리자계정(admin) 을 등록한다.

```

40      /**
41       * 관리자 등록
42       * */
43      @Test
44      @DisplayName("관리자 계정추가")
45      void memberInsert() {
46          String encPwd = passwordEncoder.encode( rawPassword: "1234"); //비번 암호화
47
48          memberRepository.save(
49              Member.builder()
50                  .id("admin")
51                  .pwd(encPwd)
52                  .role("ROLE_ADMIN")
53                  .address("오리역")
54                  .name("장희정")
55                  .build());
56      }

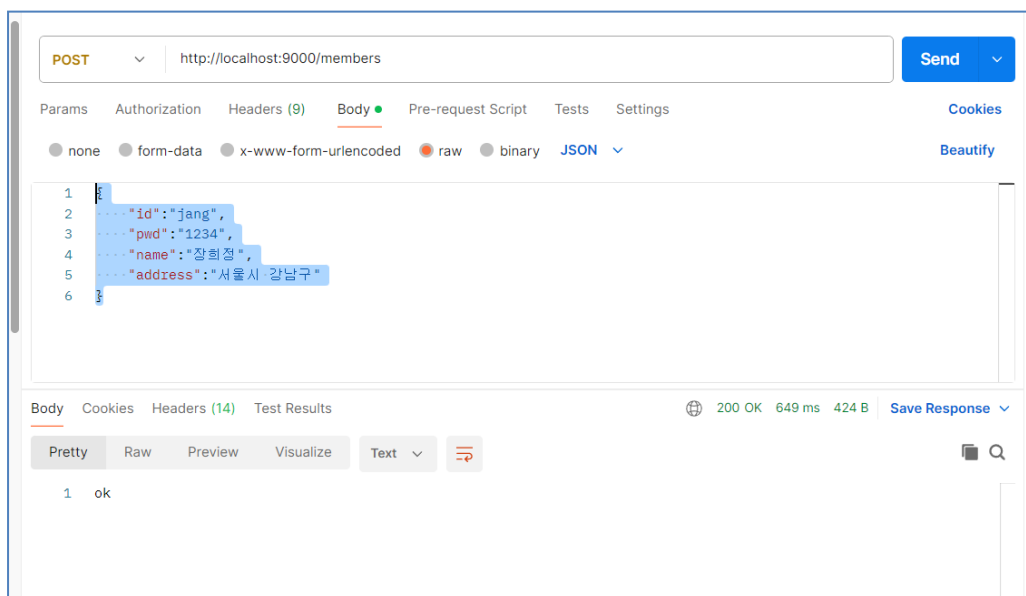
```



→ 작성한 코드를 바탕으로 회원가입, 아이디 중복을 진행한다. 모든 테스트는 PostMan을 이용한다.

1. Post Man Test

● 회원가입



json입력값 예시

```

{
  "id": "jang",
  "pwd": "1234",
  "name": "장희정",
  "address": "서울시 강남구"
}

```

POST http://localhost:9000/members

Params Authorization Headers (10) **Body** Scripts Tests Settings

☐ none ☐ form-data ☐ x-www-form-urlencoded ☒ raw ☐ binary ☐ GraphQL **JSON**

```

1 {
2   "id": "jang", → 존재하는 아이디
3   "pwd": "1234",
4   "name": "장희정",
5   "address": "서울시 강남구"
6 }
7

```

Body Cookies (1) Headers (10) Test Results 400 Bad Request 372 ms

{ JSON Preview Visualize

```

1 {
2   "type": "about:blank",
3   "title": "Duplicate Id",
4   "status": 400,
5   "detail": "아이디가 중복입니다.",
6   "instance": "/members",
7   "timestamp": "2025-06-11T22:06:47.7604638"
8 }

```

→ ProblemDetails 결과

● 아이디중복 확인

GET http://localhost:9000/members/jang Send

Params Auth Headers (9) **Body** Pre-req. Tests Settings Cookies

Query Params

Key	Value	Bulk Edit
Key	Value	

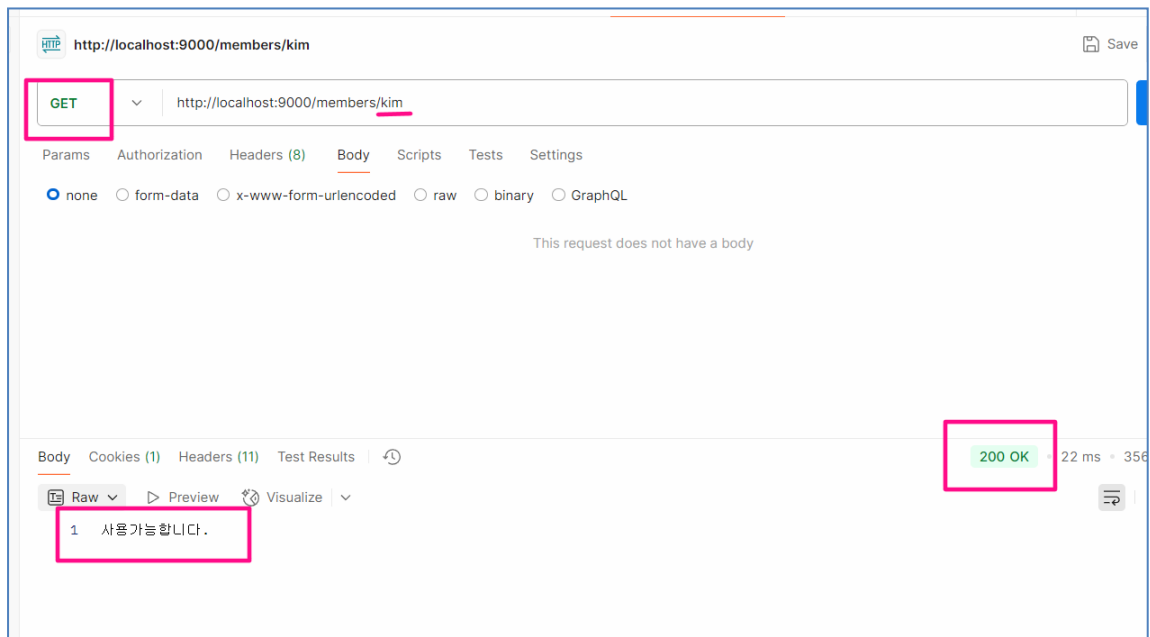
Body Cookies Headers (14) Test Results 200 OK 13 ms 439 B Save Response

Pretty Raw Preview Visualize Text

```

1 중복입니다.

```



2. 가입하기 전체흐름도

